A Way to Look at Agent Architecture

Four Planes and Workflows

Sooraj Kadnor

October 2026

Contents

1 Introduction **1**

1.1 AI Landscape 1

1.2 The Limitations 2

1.2.1 Limitations of the Models 2

1.2.2 Uncertainty for Organizations 3

1.3 What This Paper Proposes 3

2 The Architecture **4**

2.1 The Four Planes 5

2.1.1 Intelligence 5

2.1.2 Orchestration 5

2.1.3 Tools 8

2.1.4 Information 9

2.2 Workflows 10

2.2.1 Defining Workflow 10

2.2.2 Classification of Workflows 10

2.2.3 Kernel Workflow 13

2.2.4 Memory 14

2.2.5 Memory as a Workflow 18

2.2.6 Intelligence Call Control 19

3 Conclusion **22**

References **23**

1 Introduction

1.1 AI Landscape

Over time, humans have built many useful tools. Computers and the internet are among them, and artificial intelligence (AI) and AI agents are the latest.

The question behind AI is older than the technology. In 1950, Alan Turing opened his paper *Computing Machinery and Intelligence* by asking whether machines can think, and proposed a practical test, which he called the imitation game, to find out. Many others working around the same time helped popularize the idea of thinking machines. In 1955, John McCarthy, together with Marvin Minsky, Nathaniel Rochester, and Claude Shannon, proposed a summer research project at Dartmouth College. That proposal introduced the term "artificial intelligence," and the workshop held there in 1956 is widely regarded as the point where AI became a field of research.

Since then, AI has grown into the systems we now call AI models. They learn from vast amounts of data by recognizing patterns within it, and generate outputs based on what they have learned.

AI agents are the next step. An agent is a system built on top of an AI model that can reason about a goal and act to reach it. Three things set it apart from a conventional application: intelligence, provided by the model; the ability to take meaningful actions through tools, APIs, and other services; and, most important, autonomy, the capacity to decide what to do next without a person directing every step.

Organizations are adopting agents quickly. Businesses and industries are building them into:

- Customer support
- Software development
- Data analysis
- Research
- Operations

Individuals also use them directly in their own work. Gartner (a technology research and advisory firm) predicts that 40% of enterprise applications will be integrated with task-specific AI agents by the end of 2026, up from less than 5% in 2025.

But agents are not delivering as expected. Gartner predicts that over 40% of agentic AI projects will be canceled by the end of 2027, due to escalating costs, unclear business value, or inadequate risk controls. It points to a deeper cause: current models lack the maturity to autonomously achieve complex goals or follow nuanced instructions over time. Many of these problems trace back to how the models behave and to how agent systems are built around them.

1.2 The Limitations

Agents inherit the limitations of the models they are built on, and the organizations adopting them face a different kind of limitation.

1.2.1 Limitations of the Models

The generative models in widespread use today fall into two main types. Transformer models, which include the GPT series from OpenAI, Claude from Anthropic, and Gemini from Google (three companies that develop AI models), are used mainly to generate text. Diffusion models are used mainly to generate images, refining random noise into a picture over many steps. The two types are not entirely separate, since diffusion models can use a transformer inside them, as in Peebles and Xie's diffusion transformers, but they generate in different ways. Agents are built on transformer models, because those are the models that reason and produce the outputs an agent acts on, so the limitations below concern them.

These models are stateless. A model keeps nothing between calls: each call starts from zero, and everything the model should know, from its instructions and the conversation so far to the results of earlier steps, has to be sent again as part of the context. Anthropic's API documentation puts it plainly: the API is stateless, so the full conversation history is sent with every request. In today's systems, memory is something built around the model, not something the model has.

They are also autoregressive. A transformer model generates its output one token at a time, and each new token is computed from everything before it: the context and the tokens generated so far. What the model produces can only draw on what it learned in training and what is present in its context. If a fact is not in the context, the model cannot use it.

Because agents are built around these models, the two properties combine into one problem. The model needs information in its context to use it, so agents keep adding to that context: the history, retrieved documents, tool results, earlier outputs. But models use long contexts poorly. Liu and colleagues, in *Lost in the Middle*, found that models use information at the beginning and end of their context well but often overlook what sits in the middle, and also that the performance drops as contexts grow longer, even for models built for long contexts. Chroma (the company behind the open-source Chroma vector database), in its *Context Rot* report testing 18 models, found that performance grows increasingly unreliable as context length grows, even on simple tasks. This is the paradox agents run into: the model can only use what it is given, and the more it is given, the worse it uses it.

A simple task suffers from carrying context it does not need: the extra context buries what matters, and information that is present goes unused. That is the problem of the model forgetting, and the output quality falling with it. Hallucination is one way the output quality fails: the model produces content that sounds plausible but is wrong, or is not supported by the context it was given. It has several causes, including what the

model learned in training and a failure to make proper use of the information in its context. Only the second is something agent design can influence. On the other hand, a complex task has the opposite problem. It can need more context than a single call can handle well, even setting forgetting aside.

The same properties drive up cost. Because the model is stateless, the context is resent on every call, so the tokens sent grow with every turn of a conversation and every step of an agent loop. And since that context is rarely structured, much of what is resent is not needed for the call at hand. When the model loses track and gets something wrong, the work is repeated, adding to both the tokens sent and the tokens generated, and models can also pad their output with content the task does not need. Providers such as Anthropic and OpenAI offer prompt caching, which lowers the price of resending an unchanged prefix, but it does not shrink the context the model has to attend to, so the quality problem remains. All of it ends up as cost for the people running the system.

1.2.2 Uncertainty for Organizations

The second limitation is not technical. An organization deciding whether to build an agent into its workflows faces uncertainty on several fronts: what the agent will cost to run, what results it will actually deliver, whether those results justify the cost, and what risks it introduces. Costs are hard to predict because they depend on how much context each call carries and how many calls a task takes, and both vary from task to task. Results are just as uncertain, since a model that performs well in testing can degrade on the longer, messier inputs of real use. These are the same reasons Gartner gives for the cancellations noted earlier: escalating costs, unclear business value, and inadequate risk controls.

1.3 What This Paper Proposes

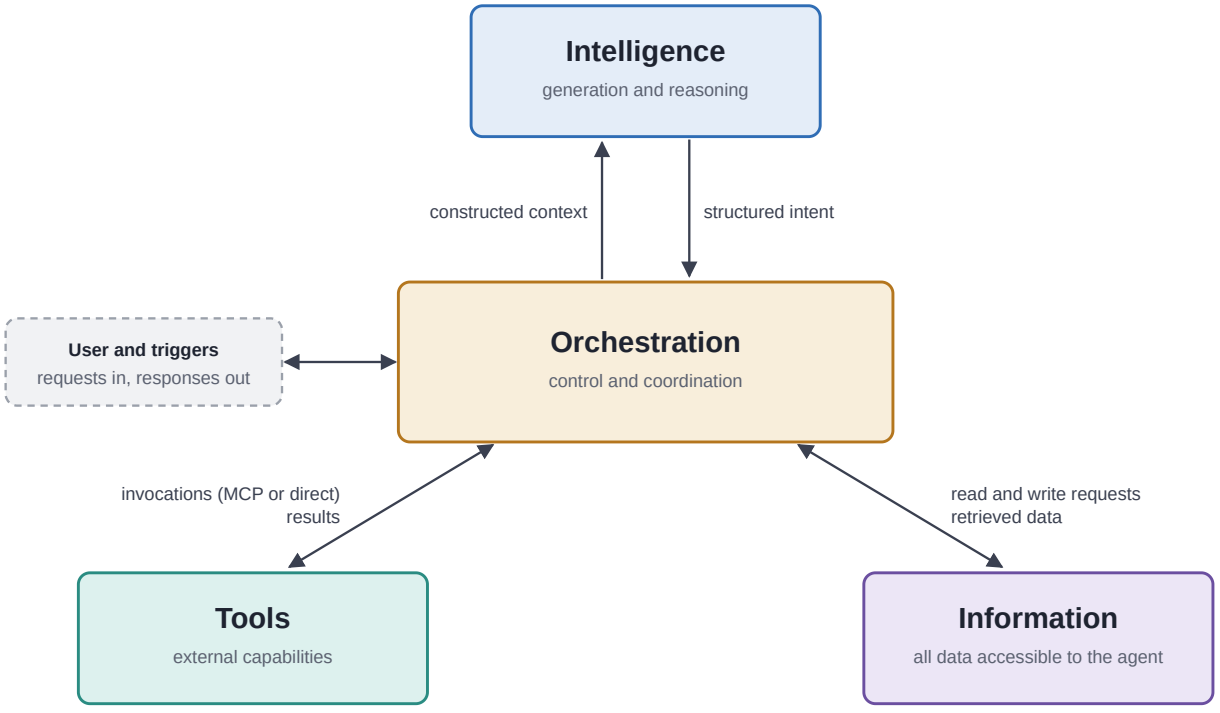
The rest of this paper sets out a way of looking at AI agent architecture that works toward solving these issues. It describes an agent in terms of four planes: Intelligence, Orchestration, Tools, and Information. Together they form what this paper calls the four-plane architecture. It also defines workflows as the processes that run across the planes. The architecture is not tied to any particular framework, model provider, or implementation.

2 The Architecture

The four-plane architecture proposed here defines four planes, each representing a fundamental aspect of how AI agents operate. Agents are predominantly built on autoregressive generative models. These models are called by agents in multiple passes, each building on the results of the previous one. This architecture is designed around that reality. The four planes, however, describe aspects that are not exclusive to generative intelligence; should agent systems evolve to incorporate other forms of intelligence, the same structural separation applies.

Each plane is defined by the function it serves. Each exists independently. Planes communicate through defined interfaces. No plane owns another plane's function. All four planes exist in parallel.

The four planes are Intelligence, Orchestration, Tools, and Information. Intelligence is connected to Orchestration. Orchestration is connected to all other planes: Intelligence, Tools, and Information. Tools and Information are each connected to Orchestration. All inter-plane communication routes through Orchestration, making it the central hub of the architecture.



Every connection runs through Orchestration. Intelligence never reaches Tools or Information directly.

2.1 The Four Planes

2.1.1 Intelligence

The Intelligence plane consists of AI models. All models here are generative by nature, and the work is probabilistic.

Generation is the fundamental function. These models operate on a single mechanical principle: given an input sequence, they predict and produce the next token, and they repeat this process until the output is complete. Every output the Intelligence plane produces, whether it is a response, a plan, a tool call request, or a memory query, is produced through this generation process.

Reasoning is what happens when this generative capability is applied iteratively, with structured context, to evaluate, analyze, and make judgments. The model reasons by generating: it produces text that breaks down problems, weighs options, critiques its own prior output, assesses which tools or models are suited for a subtask, or determines whether a task is complete. When architectures like ReAct or Reflexion feed the model's own outputs back as context with instructions to evaluate or improve them, the model is still generating, but the structured iteration of that generation produces what we recognize as reasoning. Reasoning is not a separate capability from generation; it is generation applied with purpose and iteration.

Every action in this plane follows the same pattern: the model generates structured output that represents its intent. That output takes one of several forms: a direct response to the user, a plan consisting of steps to execute, a tool call request specifying which tool with what arguments, a query requesting specific information from memory, or a control signal directed at Orchestration (such as a termination signal, an escalation request, or a model routing suggestion, among others). In this plane, the agent decides intent but does not act on it.

> **Inputs** to this plane are constructed context from Orchestration, assembled from user input, retrieved memory, tool results, and system instructions.

> **Outputs** from this plane are structured intent objects: a response to emit, a tool call to make, a memory query to execute, a plan to follow, or a control signal to Orchestration. Every output is a decision that another plane acts on.

2.1.2 Orchestration

Orchestration is the developer's domain, the control layer of the agent system. The control logic here is deterministic. The Orchestration plane is the only plane with direct connections to all other planes: Intelligence, Tools, and Information. Nothing reaches Intelligence without passing through Orchestration, and nothing leaves Intelligence without Orchestration handling it.

Control is the fundamental function. Orchestration manages how the agent system operates: what context is assembled and sent to Intelligence, what happens with Intelligence's output, when and how tools are invoked, what data is read from or written to the Information plane, and when the agent cycle ends. Every decision Intelligence produces is an intent; Orchestration is what turns that intent into action. Orchestration also runs internal code execution for its own control logic: data transformations, preprocessing, formatting, and flow control. Any code execution beyond this scope is delegated to a tool in the Tools plane.

In practice, the industry currently separates this into two layers: orchestration frameworks (which handle agent composition, workflow definition, multi-agent coordination, and task routing) and agent harnesses (which handle execution loops, tool sandboxing, context management, observability, and the operational scaffolding that makes agents reliable in production). The underlying function is the same: deterministic control and coordination. There is no architectural reason to separate them, so this architecture does not. The individual tasks within this plane can be understood as workflows, a concept defined later in this paper.

The structure of Orchestration is fluid. It must support the Intelligence plane's ability to perform complex and varied tasks, which means the orchestration logic cannot be entirely rigid. Some parts are hard-coded by the developer (fixed execution paths, known tool integrations, predetermined workflows), and some parts are adaptive, responding dynamically to whatever intent Intelligence produces. This flexibility is what allows a single agent system to handle diverse tasks rather than being locked into a single predetermined flow.

This plane manages the agent loop. When a trigger arrives (a user message, a scheduled event, a webhook), Orchestration constructs the initial context and sends it to Intelligence. When Intelligence returns a structured intent, Orchestration evaluates it. If the intent is a direct response, Orchestration delivers it to the user. If it is a tool call request, Orchestration validates the request, invokes the tool, receives the result, incorporates it into an updated context, and sends that context back to Intelligence for another generation. This cycle continues until Intelligence produces a final response or Orchestration's own constraints are met: an iteration limit, a timeout, or a budget ceiling. The loop logic, the termination conditions, and the decision of when to continue or stop all live here.

Tool invocation can happen in two ways. The first is through MCP (Model Context Protocol), which lives in this plane as the standardized protocol for dynamic tool discovery and invocation. Orchestration uses MCP to discover available tools and their schemas, includes those descriptions in the context it constructs for Intelligence, and when Intelligence decides which tool to call and with what arguments, Orchestration uses MCP to execute the invocation. This approach is suited for systems where the agent needs flexibility to work with many tools and the specific tool to call is not known in advance. The second is through direct integration, where the developer hard-codes a

specific tool call into Orchestration's logic. In this case, Orchestration already knows which tool to call for a given situation; Intelligence does not need to select it. Both approaches can coexist in the same system.

Argument validation before a tool call can also be handled in multiple ways. The developer can hard-code validation logic for a tool's expected schema, which is rigid but predictable. Alternatively, Intelligence itself can validate or properly structure the arguments through reasoning, which is more flexible and reduces the burden on the developer when tool schemas change or when the system works with many tools. A system may use both approaches depending on the tool and the situation.

When multiple agents work together, Orchestration manages their coordination. This includes handoffs between agents, shared state access, message routing, and protocols like A2A (Agent-to-Agent) for inter-agent communication. The complexity of multi-agent systems is fundamentally an orchestration problem: which agent handles which task, how state is shared without corruption, and how failures in one agent are isolated from the others.

State management across the agent's work cycle is an Orchestration responsibility. This includes checkpointing for long-running agents (so execution can be paused and resumed), managing branching logic when multiple execution paths are possible, handling parallel execution by branching tasks and merging their results, and tracking where the agent is in its current execution flow.

Error handling and recovery are central to this plane. Agents operate in unpredictable environments where tools fail, external services time out, and intermediate results arrive in unexpected formats. Orchestration handles this through retry logic with backoff for transient failures, circuit breakers that stop cascading failures before they propagate across the system, compensating actions that roll back multi-step operations when a later step fails, and fallback routing that redirects work to alternative tools or agents when the primary path is unavailable.

Orchestration captures observability data across the entire agent cycle. It logs what context was sent to Intelligence, what Intelligence returned, what tools were called with what arguments and what results came back, what data was read from or written to the Information plane, along with timing, cost, and error data. This is written to the Information plane's observability storage through a dedicated interface. Orchestration captures and emits this data; it does not analyze it.

> **Inputs** to this plane are structured intent from Intelligence, tool execution results from the Tools plane, data retrieval results from the Information plane, and external triggers (user messages, webhook events, scheduled triggers, system events).

> **Outputs** from this plane are tool invocation requests to the Tools plane (via MCP or direct integration), data read and write requests to the Information plane, constructed context packages sent to Intelligence, final responses delivered to the user, and observability event streams written to the Information plane.

2.1.3 Tools

Tools are external capabilities that the agent system can invoke but does not own. What these tools do is variable; it depends entirely on each tool. A tool might be an external API (a CRM system, a payment processor, a communication platform), a database the agent queries or writes to, a code execution environment such as a sandbox or container, a web search service, or a file system operation. Anything the agent calls to perform work it cannot do within its own Orchestration logic is a tool.

What matters about this plane is the boundary it represents. Tools exist outside the agent system. The agent does not control their implementation, does not manage their availability, and does not determine how they execute internally. From the agent's perspective, a tool is a function: it receives a structured input and returns a structured output. What happens between input and output is the tool's concern, not the agent's.

Tools can be reached in two ways, both managed by the Orchestration plane. The first is through MCP, where Orchestration discovers tools dynamically, and Intelligence selects which tool to call based on tool descriptions included in its context. The second is through direct integration, where the developer has hard-coded a specific tool call into Orchestration's logic. In both cases, the tool itself does not need to know how it was reached. It receives an invocation and returns a result.

A tool does not decide what to do with its own output. It returns structured data (a success with return values, or a failure with an error type and message) to Orchestration, and Orchestration decides what happens next: whether to pass the result to Intelligence as part of updated context, whether to call another tool, whether to retry, or whether to deliver a response to the user.

Since tools are external and outside the agent's control, they can misbehave: returning unexpected data, timing out, or producing unintended side effects. Orchestration is responsible for ensuring that such failures do not propagate into the rest of the system, through error handling, timeouts, and validation of tool outputs before they are used elsewhere.

> **Inputs** to this plane are validated invocation requests from Orchestration, consisting of the arguments required by the tool.

> **Outputs** from this plane are structured results returned to Orchestration: success with return data, or failure with error type and message.

2.1.4 Information

The Information plane is a logical boundary, not a single storage system. It represents all data accessible to the agent during its operation. It encompasses all data available to the agent, regardless of where that data physically resides or how it is accessed.

The data within this plane falls into two categories, distinguished by ownership.

Agent-generated data is data created by the agent system during its operation and used by the agent for its own functioning. This includes conversation history, structured knowledge extracted from conversations, intermediate agent states, and decision traces. This data is governed by the agent system itself: its lifecycle, access controls, and retention policies are determined by the agent's own governance. Orchestration accesses this data directly or through dedicated tools built for that purpose.

Organization-owned data is data governed by the organization. This includes data that the agent consumes during its operations, such as customer records, product catalogs, financial data, internal documents, and business databases. It also includes observability and audit data, which is generated by the agent but intended for organizational use by developers, auditors, or other systems rather than by this agent itself. The organization decides what can be accessed, how, and under what conditions. The organization provides its own tools (APIs, MCPs) for accessing this data, with access rules enforced within those tools. Orchestration connects to these tools through MCP or direct integration and operates within whatever constraints they enforce.

Both categories sit within the Information plane. The distinction is not architectural but about governance: who owns the data, who sets the access rules, who controls its lifecycle. This matters for retention, compliance, and access control.

Observability and audit data is a specific type of organization-owned data with its own dedicated interface. Orchestration captures operational data across the agent cycle (what happened, when, how long it took, what failed) and writes it here through a dedicated interface. This data has its own access controls (developers and auditors access it, not the agent itself), its own retention policies (which may span years for compliance purposes), and its own query patterns (analytical queries over operational history, not real-time retrieval for agent context).

> **Inputs** to this plane are read and write requests from Orchestration, governed by the access rules of whichever category the data belongs to.

> **Outputs** from this plane are retrieved data sent to Orchestration, subject to any access restrictions enforced by the governing entity of that data.

2.2 Workflows

2.2.1 Defining Workflow

A workflow is a structured, executable process that coordinates tasks, resources, and dependencies to accomplish a defined function.

Every workflow has a purpose (the function it accomplishes), a trigger (what initiates it), a structure (the sequence, branching, looping, or parallel execution of tasks), state (awareness of what has been done and what remains), and a termination condition (how it ends: completion, failure, or timeout).

Tasks are the individual units of work within a workflow. Each task is a single operation: call Intelligence for a response, call a tool, validate an argument, choose between branches, retrieve data, write to storage, format an output, or wait for a person's approval. Resources are whatever the tasks need to operate: compute, data, tools, models, human input. Dependencies are the relationships between tasks that determine their order and conditions: what must happen before what, what can happen simultaneously, and what is conditional on what.

Workflows are composable. A workflow can invoke other workflows as steps within its own execution, and this composability is recursive: a sub-workflow can itself invoke further sub-workflows.

The four planes described earlier are architectural concepts; they define what kind of work happens where. Workflows are the processes that put those concepts into practice. Intelligence contributes reasoning and generation, Orchestration contributes control and coordination, Tools contribute external actions, and Information represents the range of data the agent deals with. Workflows use these capabilities to accomplish specific functions, and a single workflow may stay within one plane or cut across several, depending on what it needs to do.

2.2.2 Classification of Workflows

Workflows can be classified along two independent axes.

The first axis is where the workflow executes relative to the agent's Orchestration plane. Internal workflows execute within it under direct developer control. The workflow's logic, sequencing, error handling, and state management are all part of the

orchestration code the developer writes. External workflows exist outside it and are accessed through tool calls. Orchestration triggers them and receives results, but the workflow's internal logic runs elsewhere.

The second axis is whether the workflow is agentic or non-agentic. An agentic workflow operates using the planar architecture. It is an agent in itself: it has its own Orchestration managing the flow, its own access to Intelligence for reasoning, and whatever other planes it needs. An agentic workflow can reason, make decisions, and adapt to situations that were not explicitly pre-programmed.

A non-agentic workflow is pre-coded logic. A classical program, script, or set of instructions. On its own, it follows predetermined logic, does not use the planar architecture, and can only handle situations that were explicitly programmed.

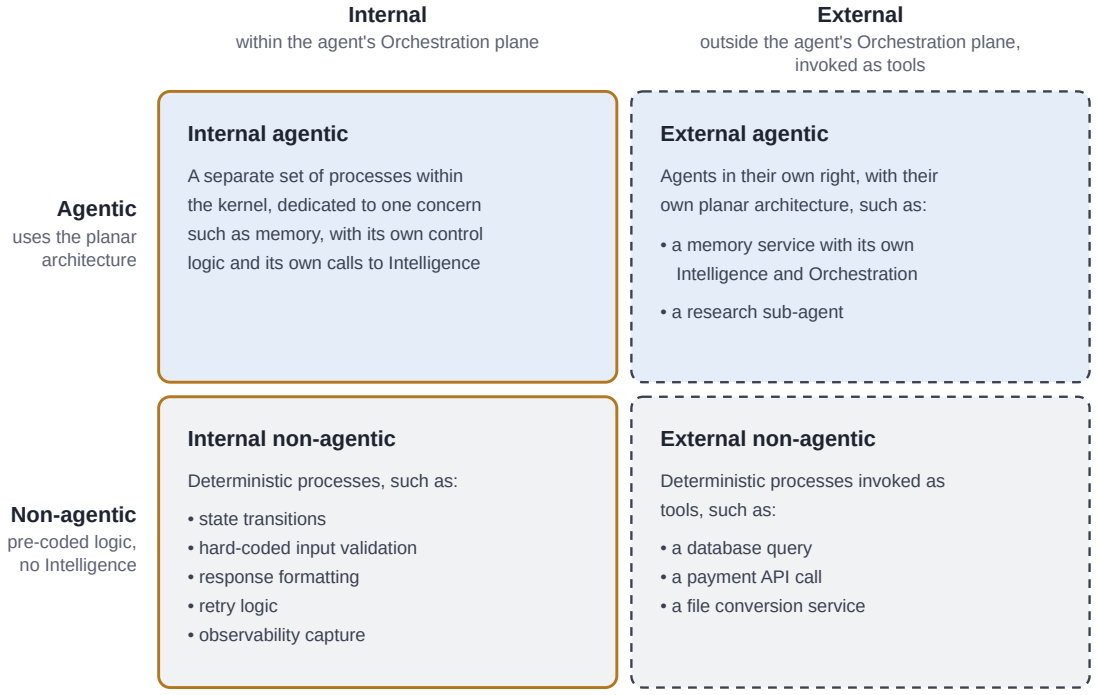
A script that calls an LLM API shows where the boundary lies. The script itself is a non-agentic workflow: it sends data to the model and takes the response back. The model's generation is Intelligence, and the script together with that generation can form an agentic workflow, but the script alone stays non-agentic.

Non-generative models, such as classifiers or discriminative models, sit on the other side of that boundary. They are not part of the Intelligence plane: they receive input, produce a fixed output, and return it, without reasoning, planning, or generating structured intent. A script that calls one remains non-agentic, and when an agent system uses a classifier, it is invoking a non-agentic workflow, either as a tool or as a step within Orchestration.

The distinction is not about complexity. A non-agentic workflow can be enormously complex. It is about whether the workflow operates as an agent with the planar architecture or executes as predetermined code.

The design principle: use agentic workflows only where the task requires reasoning, judgment, or adaptation to unpredictable inputs. Use non-agentic workflows for everything else.

These two axes produce four quadrants.



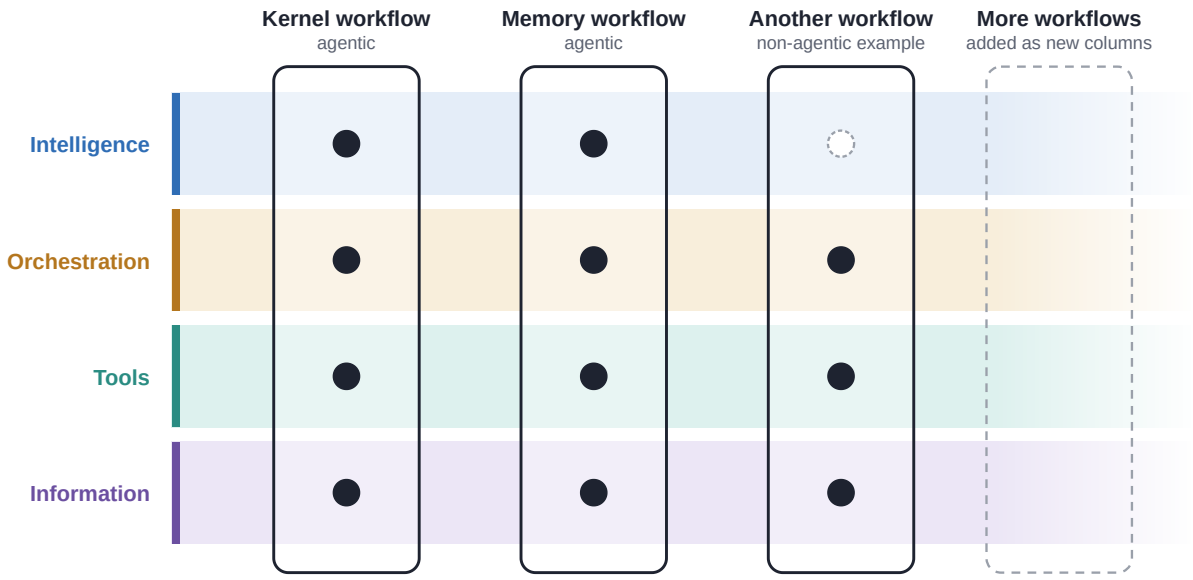
Solid amber border: runs inside the agent's Orchestration plane. Dashed border: runs outside it. Blue fill: agentic.

Internal agentic workflows are agent processes that execute within the agent's Orchestration plane. An example is a separate set of processes within the kernel, dedicated to a distinct concern such as memory. These processes carry their own control logic and can call Intelligence for that purpose, while remaining part of the same agent system.

Internal non-agentic workflows are deterministic processes within the agent's Orchestration plane. Examples include state transitions, hard-coded input validation, response formatting, retry logic, and observability capture. They follow predetermined logic and produce predictable outputs.

External agentic workflows are agent processes that execute outside the agent's Orchestration plane, invoked as tools. These are agents in their own right, with their own planar architecture. A memory service with its own Intelligence and Orchestration is an external agentic workflow, and so is a research agent that searches and synthesizes on its own. From the kernel workflow's side, each one is a tool: it receives a request and returns a result.

External non-agentic workflows are deterministic processes outside the agent's Orchestration plane, invoked as tools. A database query, a payment API call, a file conversion service.



A filled dot marks a plane the workflow uses. A non-agentic workflow never uses Intelligence. The planes extend to the right: the architecture stays the same as new workflows are added.

With the workflow concept and its classification defined, the following sections describe some of the major workflows in an agent system: the kernel, memory, and intelligence call control.

2.2.3 Kernel Workflow

In an operating system, the kernel is the core program that manages all system resources, mediates all interactions between hardware and software, and remains active for as long as the system is running. The kernel workflow in an agent system serves a similar role.

It is the core operational workflow of an agent system, what the developer builds to manage and coordinate the agent's entire operation. The Orchestration plane is where control and coordination happen, and the kernel workflow is what makes that plane operational. When a developer builds an agent, the kernel workflow is what they are building.

The kernel workflow manages the agent loop. It receives triggers, constructs context, calls Intelligence, evaluates the output, acts on it, and determines when the cycle ends. It manages access to all planes: nothing reaches Intelligence without the kernel workflow sending it, no tool is invoked without the kernel workflow triggering it, no information is read or written without the kernel workflow initiating the request. Other workflows run at its call: it invokes them when needed, supplies their inputs, and handles their results. It also enforces boundaries, such as iteration limits, timeout constraints, budget ceilings, and the access policies that govern the agent's behavior.

The kernel workflow does not do the reasoning, which is Intelligence's job, and it does not run tool logic, which belongs to the Tools. Its role is to coordinate them.

The kernel workflow is always agentic, since it uses the planar architecture: Intelligence for reasoning, Orchestration for control, and access to Tools and Information as needed.

Its sub-workflows are both agentic and non-agentic. Some steps are pre-coded logic the developer wrote: validation, formatting, routing rules, deterministic state transitions. Others invoke agentic workflows, such as the memory workflow, a research workflow, or a planning workflow. The kernel is the top-level composition: every other workflow either runs inside it or is called from it.

Separating a workflow from the kernel is a development and architectural choice. All workflows could exist inside the kernel. Separating them provides the same benefits as separating features into independent services in traditional software development: independent development, clear interfaces, and the ability to swap implementations without affecting the rest of the system. Two workflows connect through defined interfaces, just as two services connect through APIs.

2.2.4 Memory

Memory is the capability of an agent system to manage information through encoding, storage, retrieval, and context generation. Its purpose is to provide the necessary information to the Intelligence plane as and when needed.

Memory draws information from across the Information plane, within the limits of the policies that govern it. Whether the agent needs a fact from a past conversation or a customer record from the organization's systems, memory retrieves it and delivers it where it is needed, as long as those policies allow it. Memory does not distinguish between agent-generated and organization-owned data when assembling what the agent needs; it brings together whatever is necessary and permitted for the current operation.

What differs between the two is who sets the policy. Agent-generated data is governed by the agent system's own policies, and memory writes to it freely: conversation history, extracted knowledge, decision traces, intermediate states. Organization-owned data is governed by the organization's own policies, enforced through the tools the organization provides, as defined in the Information plane. Those policies can restrict reading as well as writing, and where they deny access, memory cannot retrieve the information.

Memory is not the same thing as the Information plane. The plane is a logical boundary describing all the information the agent deals with, while memory is a capability that operates across that information: selecting what is relevant, organizing it for use, and maintaining it over time. One describes what exists; the other determines what the agent actively works with.

Levels of Memory

Memory organizes information across three levels, based on how immediately it is being used.

Context is the most immediate level: the constructed input that the Intelligence plane receives for a single generation. Orchestration assembles it from the information currently present in working memory. Context exists only for the duration of one inference call. It is assembled, not stored; once Intelligence generates its output, that specific context has been consumed.

Context is ephemeral by default. Orchestration can choose to persist specific contexts when auditability or debugging requires it, but the architecture does not mandate this. Context is included as a level of memory because the way it is constructed directly affects agent performance. Models degrade as context grows, even within their window limits; irrelevant content pulls attention away from what matters, and poorly organized context produces poor output. Treating context as a managed level means handling it as the constrained resource it is.

Working memory sits in between. It holds the state that persists across steps within a single agent work cycle: where the agent is in its current task, what has happened so far, what intermediate results exist, and what comes next. Orchestration manages it throughout the agent's work cycle.

Working memory is volatile. When the work cycle ends, it is gone unless explicitly saved to persistent memory. If the agent fails mid-task and working memory was never checkpointed, everything from that execution is lost. Checkpointing bridges working memory and persistent memory: Orchestration periodically needs to write the current state to the Information plane, creating recovery points. The frequency and conditions of checkpointing are engineering decisions balancing the cost of losing state against the cost of persisting it.

Each time Orchestration calls Intelligence, it constructs context from working memory. If additional information is needed from persistent memory or tools, Orchestration retrieves it into working memory before constructing context. When Intelligence returns its output, Orchestration updates working memory with the result: what was decided, what tool was called, what was returned. This cycle of updating working memory and constructing context from it is what gives the agent coherence across multiple passes.

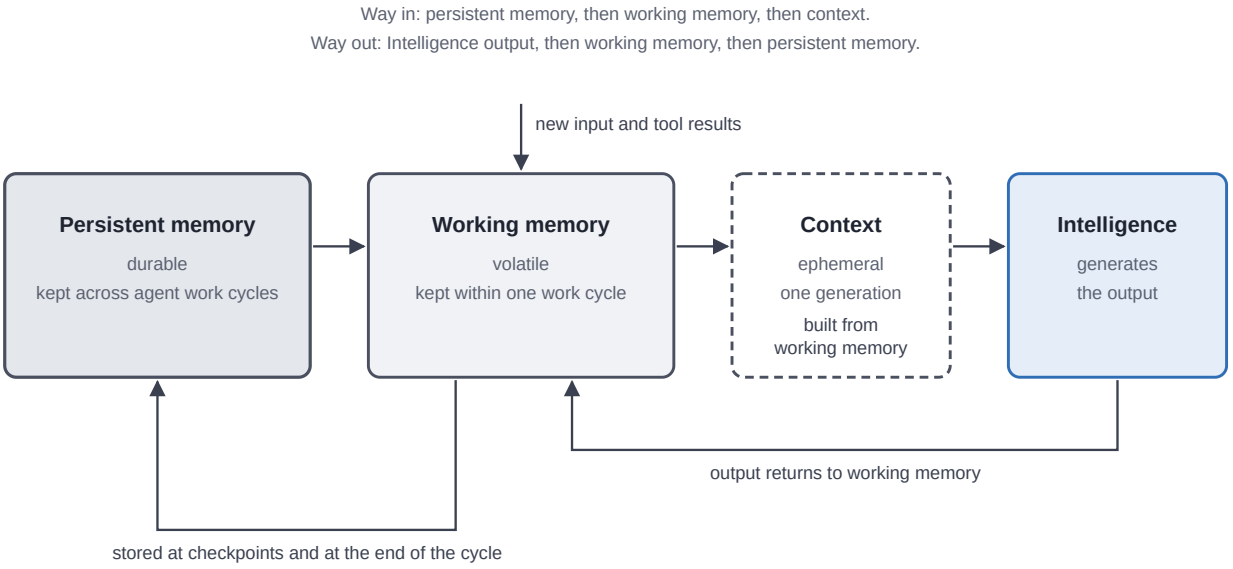
Persistent memory is the most durable level. It is information retained in the Information plane across agent work cycles, such as conversation history accumulated over sessions, structured knowledge extracted from conversations, decision traces from past operations, and organization-owned data the agent can access. Persistent memory is durable and large, but slower to access than working memory or context.

Retrieving from it requires a retrieval call, a database query, or a search operation. Orchestration must retrieve selectively because not everything in persistent memory can or should be loaded into context at once.

The governance of persistent memory depends on the category of data. Agent-generated persistent memory is governed by the agent system's own policies: retention periods, access controls, consolidation rules. Organization-owned persistent memory is governed by the organization, which enforces its access rules in the tools it provides.

How Information Flows Between Levels

The flow on the way in is: persistent memory to working memory to context. The flow on the way out is: Intelligence output to working memory to persistent memory. This cycle is continuous throughout the agent's operation, and the memory workflow is the process that manages it.



Memory Processes

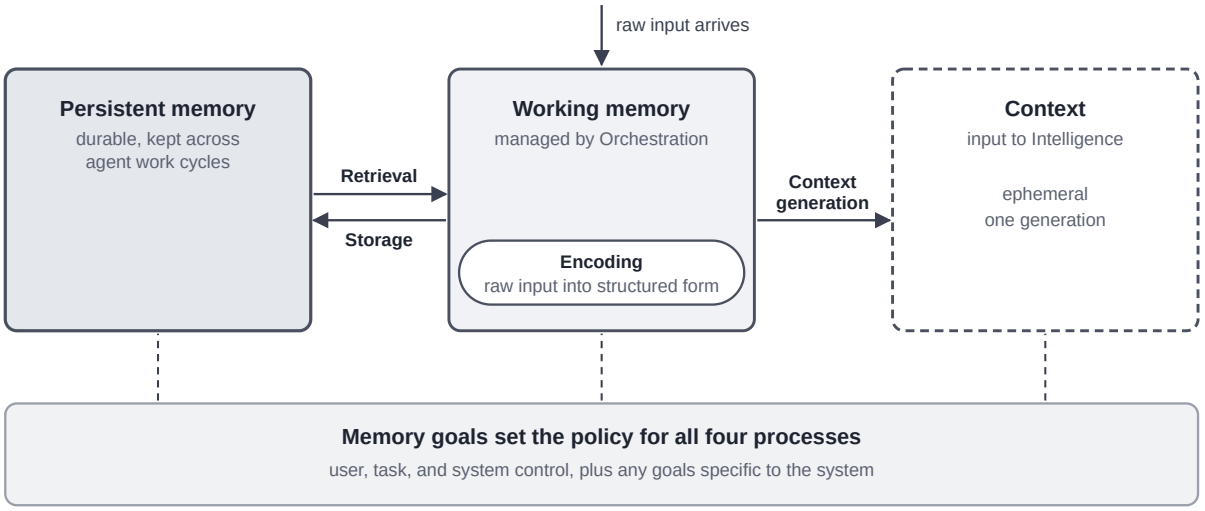
The memory capability operates through four processes.

Encoding is the organization of raw information into structured form for storage. When raw data arrives (a conversation message, a tool result, a document), encoding processes it: extracting meaning, generating metadata, identifying relationships, and structuring it into a usable format.

Storage is persisting information from working memory to persistent memory, the act of making volatile information durable. What gets stored, when, and under what conditions are policy decisions guided by the memory goals. Storage includes consolidation as a sub-process: resolving contradictions in stored information, deduplicating, summarizing older data, and enforcing retention policies to maintain the quality of persistent memory over time.

Retrieval is bringing relevant information from persistent memory into working memory, the reverse of storage: making durable information available for active use. As with storage, the memory goals set the policy here: what gets retrieved, how much, and how it is selected. Retrieval follows the access policies applicable to the data being retrieved, whether agent-generated or organization-owned.

Context generation is assembling information from working memory into the constructed input for the Intelligence plane. It takes what is currently available in working memory and structures it into the specific context that Intelligence will receive for a single generation call. Context generation determines what information the model receives, how it is organized, and how it fits within the constraints of the context window.



Memory Goals

Goals guide the policies of how encoding, storage, retrieval, and context generation behave. They are design decisions made by the developer or the organization that determine the priorities and constraints behind those policies. Goals are specific to the system being built. They emerge from what the system is meant to accomplish and what constraints it operates under.

Most agent systems will need to address three primary goals.

User means memory operates to serve the user's experience. Remember who the user is, maintain continuity across sessions, retain preferences, and avoid making the user repeat themselves. This goal drives policies around what user information to retain, how aggressively to retrieve user context, and how to prioritize user-relevant information in context generation.

Task means memory serves the current task. Provide the right information to facilitate what the agent is doing right now. Here the policies cover task-relevant retrieval, working memory coherence across steps, and checkpointing at task milestones.

System control means memory operates within the constraints of the system itself. This has three dimensions. System resources: memory must respect token budgets, storage limits, latency requirements, and cost constraints. Not everything can be loaded into context. Not everything can be stored forever. Retrieval must be fast enough not to bottleneck the agent loop. Multi-agents: when multiple agents share a system, memory must handle concurrent access, shared state, isolation between agents, and conflict resolution. Distributed computing: when the agent system runs across multiple machines or environments, memory must handle synchronization, consistency, and availability across distributed infrastructure.

User and task are purpose goals: they define what memory is optimizing for. System control is a constraint goal, setting the boundaries memory has to stay within while it pursues those two.

These three are not exhaustive. The developer should consider what their specific system is optimizing for, what constraints it operates under, and what external requirements (regulatory, organizational, security) apply. The goals that emerge from that analysis govern memory policy.

2.2.5 Memory as a Workflow

The memory workflow is how the capability of memory is implemented. It manages the movement of information between persistent memory, working memory, and context, executing the four memory processes (encoding, storage, retrieval, context generation) according to the policies set by the memory goals.

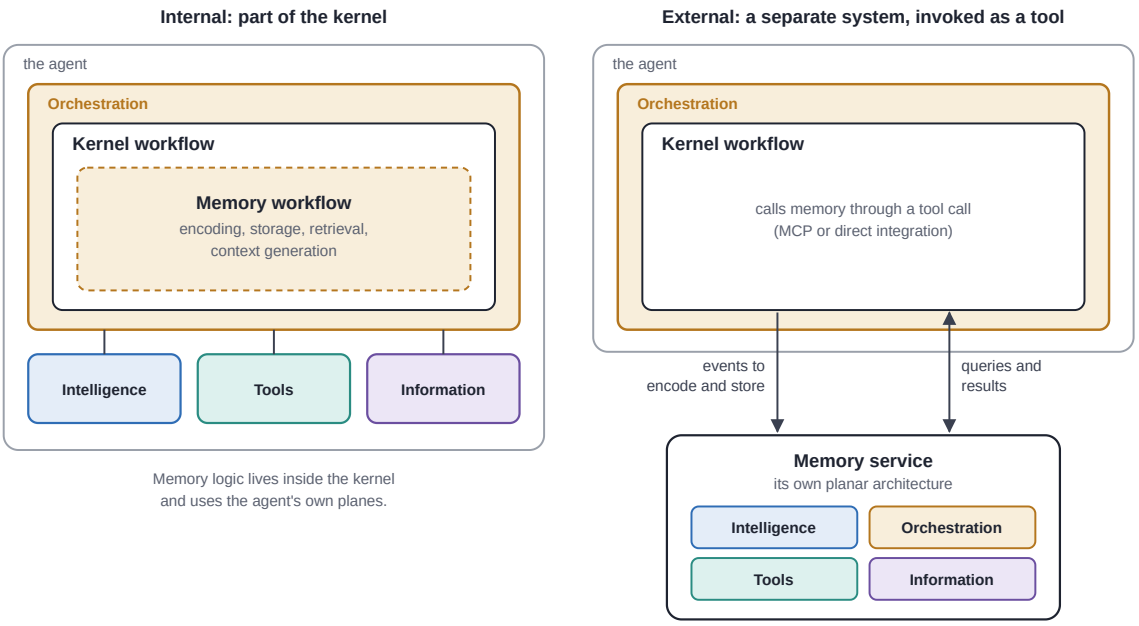
It operates across all four planes. Intelligence for reasoning about information: understanding raw data, extracting structured knowledge, identifying relationships, resolving ambiguities. Orchestration for control: deciding when to retrieve, what to encode, when to persist, and how to assemble context. Tools for the external services it uses, such as search, database access, and storage writes. Information as the range of data it works across, at both the working and persistent levels.

Because the memory workflow uses the planar architecture, it is an agentic workflow.

The memory workflow is a segment of the kernel workflow. When internal, its orchestration logic is part of the kernel directly. The developer builds memory management into their agent's core operation. This is simpler to start with but means the developer carries the full complexity of memory management within their own codebase.

When external, the memory workflow is a separate system with its own planar architecture, invoked by the kernel workflow as a tool. The kernel sends events to the external memory service (for encoding and storage) and queries it (for retrieval and

context generation). The external service handles the complexity independently. The kernel treats memory as a capability it consumes, not one it builds.



Either approach is valid, and the choice depends on the complexity of the memory requirements, the development resources available, and whether the developer wants to build and maintain the memory logic themselves within the kernel workflow or use dedicated memory infrastructure.

2.2.6 Intelligence Call Control

Intelligence Call Control governs how the kernel workflow invokes the Intelligence plane. It does not own context generation (that remains a memory process). What it determines is the structure of the call: whether a single call is made or several, how those calls relate to each other, and what each call is scoped to do. It is orchestration-plane work, a segment of the kernel workflow that can live inside the kernel or be separated out, following the same principle as the memory workflow. Once it has set the structure, Orchestration requests from the memory workflow the context each of those calls needs.

This workflow matters because how a call is structured affects both token cost and the reliability of the output. One large, undifferentiated prompt makes the model attend to everything at once, and as the context grows, its focus dilutes, irrelevant context competes with relevant context, and drift becomes more likely. Structuring how calls are made, whether by splitting a large task into narrower targeted calls, routing different inputs to different handling paths, or running independent sub-tasks in parallel, reduces both the cost of each call and the likelihood of the model losing track of its objective.

The Baseline: A Direct Call

The simplest and most common way to invoke the Intelligence plane is a direct call. The system prompt, the full conversation history, the current user message, tool definitions, and any orchestration-injected information (retrieved memory, tool results, prior reasoning outputs) are compiled into a single context and passed to the model. The model generates a completion: it continues the text based on everything in that context. The result is returned to Orchestration.

The model is stateless between calls. It has no memory of prior invocations. Conversation history persists only because Orchestration resends it as part of the constructed context each time. Every call is, from the model's perspective, the first and only call it has ever seen.

Structured Approaches Beyond the Baseline

When a direct call falls short, whether because the task is too complex for one pass, the context is too large for the model to handle reliably, or the output needs checking, Orchestration can add structure to how Intelligence is called. The techniques below are some of the established ways to do this, drawn from Anthropic's *Building Effective Agents* and, for plan-and-execute, LangChain's *Plan-and-Execute Agents*.

Prompt chaining breaks a task into a sequence of calls, each working on the output of the one before. Instead of one call handling the whole task, each call handles a single stage of it (for example: draft an outline, check it against criteria, then write the document), and checks can sit between stages to confirm the work is still on track. It suits tasks that divide cleanly into fixed stages. Each call has a narrower job and is easier to get right, at the cost of added latency.

Routing classifies the input first and then sends it down the path suited to that kind of input, whether that means a different prompt, a different model, or a different workflow (for example: simple questions to a small, fast model and complex ones to a larger model). The classification itself can be done by a call to Intelligence or by a conventional classification model. It suits systems that receive distinct kinds of input, since each path can be tuned without hurting the others, provided the classification is accurate.

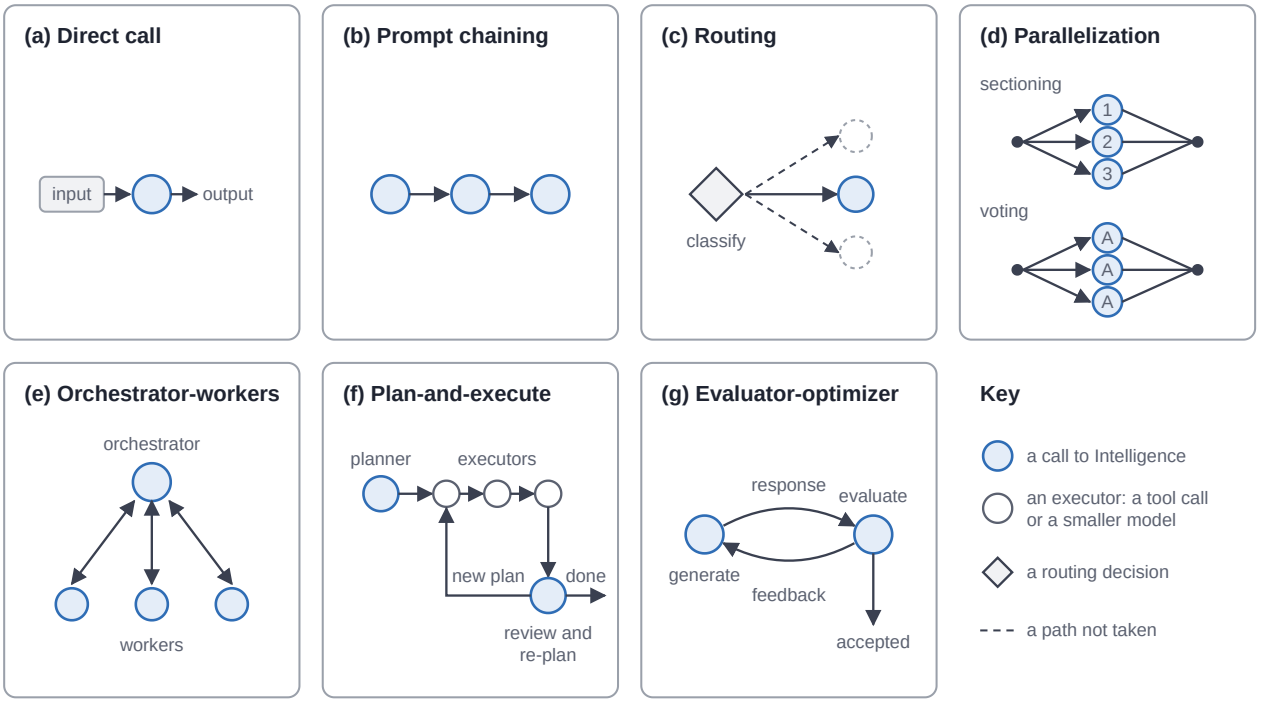
Parallelization runs several calls at the same time and combines their outputs. It takes two forms. In sectioning, a task is split into independent parts handled side by side (for example: researching different aspects of a question at the same time, with one call for each aspect). In voting, the same task is run several times and the results are compared (for example: several reviews of the same code for vulnerabilities). It suits parts that do not depend on each other, or work where agreement across attempts raises confidence, at the cost of more calls.

Orchestrator-workers uses one call to divide the work and other calls to do it. A central call reads the task, decides which sub-tasks it needs, passes each to a worker call, and combines what they return. Because the sub-tasks are decided for each input,

it suits tasks whose parts cannot be predicted beforehand (for example: a code change where the files involved depend on the request). The orchestrator here is a call to Intelligence, not the Orchestration plane.

Plan-and-execute separates deciding what to do from doing it. A planner call first writes a step-by-step plan for the whole task. Executors then carry out the steps, often as direct tool calls or with a smaller model, and when they finish, a re-planning call decides whether the task is done or needs a new plan. Because the larger model plans once instead of being consulted after every action, it saves cost and time on long, multi-step tasks. A flawed plan, however, carries its error into the steps that follow until re-planning catches it.

Evaluator-optimizer pairs a call that produces work with a call that judges it. One call generates a response, a second evaluates it against defined criteria and gives feedback, and the first revises, repeating until the evaluator accepts the result or a limit is reached. It suits work that clearly improves with feedback (for example: literary translation, where an evaluator can catch nuances the translator missed). It needs clear criteria to be useful, since every round adds calls and latency.



These techniques can be built on one another. Plan-and-execute builds on prompt chaining, and orchestrator-workers builds on parallelization, with the structure worked out for each input instead of being set in advance. Routing and evaluator-optimizer can be placed around any of the others. Together, they allow a structure as complex as the task needs. As with agentic and non-agentic workflows, the simplest structure that works is the right one: a direct call is the default, and more structure is added only when the simpler approach falls short.

3 Conclusion

The introduction set out the limitations this architecture is meant to address. The models on which agents are built are stateless and autoregressive, which pushes agents to keep resending and enlarging their context even though models use long contexts poorly. Output quality falls and costs rise as a result. Organizations, meanwhile, face uncertainty about what an agent will cost, what it will deliver, and what risks it brings. Each of these is handled by a specific part of the architecture.

Start with statelessness. The answer is to treat memory as a capability of the agent system rather than something expected of the model. The memory workflow keeps information at three levels: persistent memory across work cycles, working memory within a single work cycle, and context for a single call. The model still starts every call from zero, but the agent no longer has to resend everything it knows to make up for that.

The paradox, that a model can only use what it is given yet uses large contexts poorly, is where context generation comes in. Instead of placing all available information into one growing context, the memory workflow builds each context from working memory, carrying what the current call needs and leaving out what it does not. Retrieval brings in only the relevant parts of persistent memory. The memory goals, including the system's own resource limits, decide what counts as relevant and how much is brought in. A smaller, focused context makes it less likely that the information that matters gets overlooked.

Intelligence call control takes the other side of the paradox: tasks too complex for a single call. It splits the work across several calls, whether as a fixed sequence, parallel calls, a plan carried out step by step, or sub-tasks decided for each input, so each call gets a narrower job and a context that fits it. The two workflows work together: intelligence call control sets the structure of the calls, and memory supplies the right context for each one.

Cost comes down through the same mechanisms. Smaller, structured contexts mean fewer tokens per call, and because past work lives in persistent memory, the context does not have to grow with the length of a session. Better context cuts down the errors that cause repeated work, and checkpointing working memory lets a failed task resume from its last saved state instead of starting over. Scoping each call to a single job also leaves less room for padded output.

For organizations, the architecture's answer to uncertainty is to make the agent's behavior bounded and visible. The kernel workflow can enforce limits on iterations, time, and budget, so the cost of a task can have a ceiling set in advance. Orchestration captures what happens across the agent cycle (what context was sent, what came back, which tools were called, and what it cost), giving the organization a record to judge

results and risks against. And because the architecture divides the agent into planes and workflows, each part can be examined, measured, and improved on its own, instead of the agent being treated as one opaque system.

The four-plane architecture proposed here is a way of looking at agent systems, not a specification. It is offered as a starting point for developers and architects building production agents, and as a proposal that others are free to adopt, challenge, and improve.

References

Anthropic. (2024). *Building Effective Agents*. Anthropic Engineering Blog.

Anthropic. *Prompt Caching*. Claude API Documentation.

Anthropic. *Using the Messages API*. Claude API Documentation.

Chroma. (2025). *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Technical report.

Gartner. (2025). *Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026, Up from Less Than 5% in 2025*. Press release.

Gartner. (2025). *Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027*. Press release.

Google. (2025). *Announcing the Agent2Agent Protocol* (A2A). Google for Developers.

LangChain. (2024). *Plan-and-Execute Agents*. LangChain Blog.

Liu, N. F., et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics.

McCarthy, J., Minsky, M. L., Rochester, N., & Shannon, C. E. (1955). *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*.

Model Context Protocol. *Model Context Protocol Specification*.

OpenAI. *Prompt Caching*. OpenAI API Documentation.

Peebles, W., & Xie, S. (2023). *Scalable Diffusion Models with Transformers*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV).

Shinn, N., et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. Advances in Neural Information Processing Systems (NeurIPS).

Turing, A. M. (1950). *Computing Machinery and Intelligence*. Mind.

Yao, S., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR).